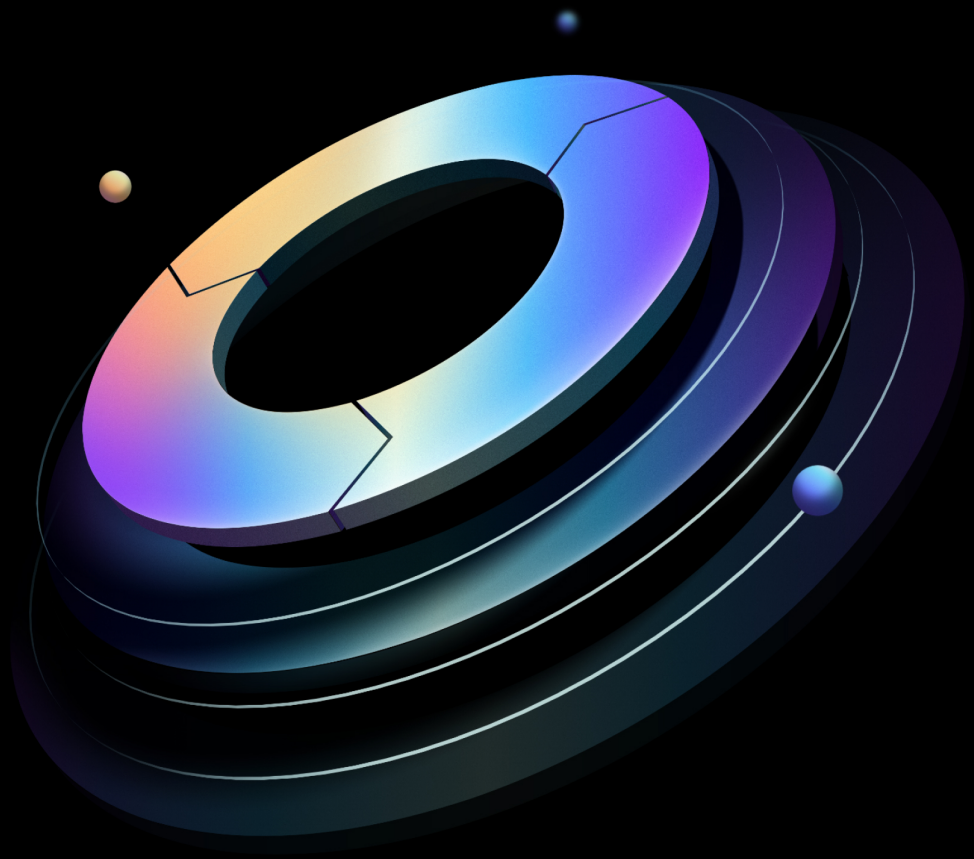
HashiCorp

Vault Radar User Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **2**
1.1 Why use HashiCorp Validated Designs? . . . 2
1.2 What this guide covers . . . 3
1.3 Related guides . . . 4

2 Deployment models **4**
2.1 Software as a service (SaaS) . . . 4
2.2 Hybrid deployment with agents . . . 4
2.3 Automation pipelines . . . 5
2.4 Multi-model example . . . 5

3 Using Vault Radar in pipelines **5**
3.1 Pre-commit hook (client side) . . . 6
3.1.1 Prerequisites . . . 6
3.2 Pre-receive hooks (git server) . . . 7
3.2.1 Prerequisites . . . 7
3.3 Pull request scanning . . . 10
3.4 CI/CD scanning . . . 10
3.4.1 Prerequisites . . . 10
3.5 Running Vault Radar CLI scan and meter commands . . . 11
3.6 CLI indexing commands . . . 12

4 Remediation workflows **12**
4.1 Remediation recommendations . . . 13
4.2 Remediation actions . . . 14

5 Changelog **15**
5.1 2026-08 . . . 15

6 Credits **15**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

Note

This guidance for HCP Vault Radar is *emerging guidance*. We continually update it as we identify common best practices and configurations to help customers get the most from the product.Throughout this guide, the terms HCP Vault Radar, Vault Radar, and Radar are interchangeable. There is no self-managed or enterprise version of Vault Radar.

Vault Radar's purpose is to help inspect, protect, and govern environments by automating the detection and identification of unmanaged secrets in code and other locations so that security teams can take appropriate actions to remediate issues. It continuously scans in real-time for secrets.

Vault Radar scans connected cloud and on-premise data sources, detecting over 300 secret patterns across a large number of data sources. Radar automatically scans data sources when they are initially added, and also when there are new commits and new pull requests.

It provides severity, source, type, and other details to help you prioritize and remediate insecure secrets.

Vault Radar also provides the following benefits:

- Supports easy hand-off to developers and code repository owners for follow-up either in the application or via notification integrations
- Can warn or block pull requests when it finds risk in code
- Prevents secrets and sensitive information appearing in code, before and during CI activities
- Progress reporting

Tip

Vault Radar does not send or store source code or sensitive data. Instead, Vault Radar performs a two-phase hash or peppering so Vault Radar can identify if the sensitive data exists in different data sources. This hash is then tokenized and returns a universally unique identifier (UUID) which Vault Radar stores in the HashiCorp Cloud Platform. The generated UUID, the commit hash, and the line number where Vault Radar finds the sensitive data is available in the HCP Portal.

1.2 What this guide covers

This User Guide is for the developers and application teams who *consume* Vault Radar:

- **Deployment models** — how Radar scans SaaS, hybrid (agent), and pipeline sources.
- **Using Vault Radar in pipelines** — pre-commit hooks, pre-receive hooks, pull request scanning, and CI/CD scanning.
- **Remediation workflows** — acting on the findings Radar surfaces.

Vault Radar helps Security Operations (SecOps) teams with automated processes, risk detection rules and repeatable remediation steps. Using it well allows your organization to:

- **Accelerate SecOps agility**: Centralize monitoring of events and quickly identify insecure secrets
- **Streamline remediation processes**: Efficiently triage security issues and seamlessly hand them off to development teams
- **Prevent security breaches**: Detect and eliminate leaked secrets in code repositories, collaboration tools, and other insecure locations

- **Reduce business risk**: Proactively secure sensitive credentials
- **Minimize operational costs**: Prevent application downtime and security incidents caused by compromised secrets

1.3 Related guides

Vault Radar is configured and governed by platform operators. For roles, access management, data source onboarding, agent deployment, and alerting configuration, see the Administration Guide.

2 Deployment models

Vault Radar offers multiple deployment models to accommodate your organizational needs and infrastructure requirements. Each model addresses specific use cases, from public cloud resources to private networks and development workflows. Understanding these deployment options helps you design a comprehensive secrets detection strategy. We recommend using a mix of all methods, especially if you have hybrid data source infrastructure.

2.1 Software as a service (SaaS)

Using the HashiCorp Cloud Platform (HCP), scanning publicly available data sources such as GitHub Cloud, Confluence, and other services for risks. This model allows you to set up secrets detection quickly, but works best for data sources accessible via the internet.

2.2 Hybrid deployment with agents

If you have resources that are not publicly accessible by HCP, use hybrid scanning inside your private network. This model requires installation of Vault Radar agents and agent pools. There are two ways to do this:

1. Vault Radar agents
2. Vault Radar CLI and upload to HCP

Deploying and scaling agents is an operator task — see the Administration Guide: Agent deployment.

2.3 Automation pipelines

Vault Radar can scan incoming pull requests, development environments and CI/CD pipelines. This provides automatic preventative alerting in the case of new secret identification, and it then block commits and pull requests. For how to set this up in your workflows, see Using Vault Radar in pipelines.

2.4 Multi-model example

In the following diagram Vault Radar scans publicly available data sources (SaaS), and self-hosted data sources (hybrid) using an agent pool.

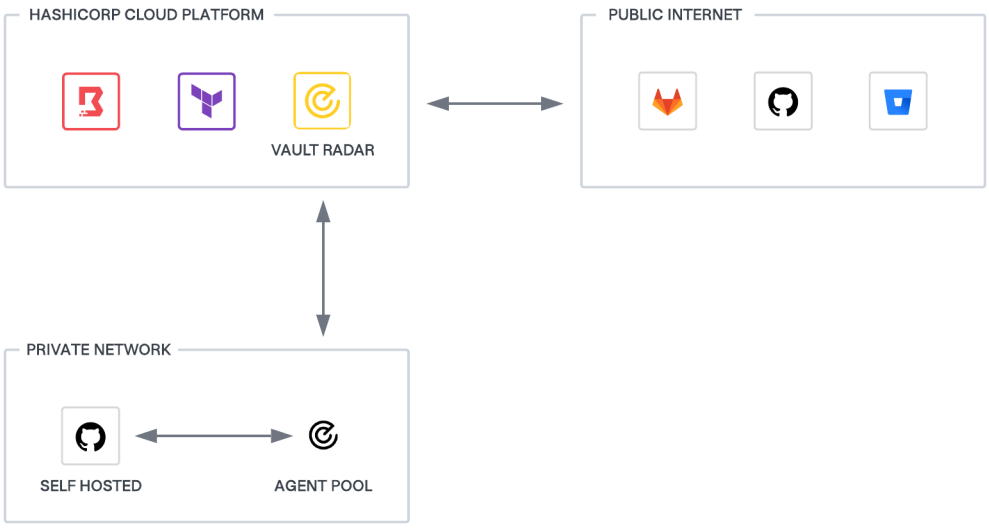


Figure 1: Vault Radar deployment models diagram

3 Using Vault Radar in pipelines

Vault Radar stops new secrets ending up in the wrong hands, supporting developer workflows by scanning prior to code commits to repositories.

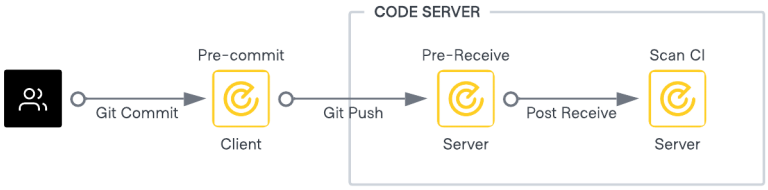


Figure 2: Vault Radar automation workflow

3.1 Pre-commit hook (client side)

Use pre-commit configuration as a first layer of protection for commits.

> Tip
>
> For larger scans, using a pre-commit hook is preferable.

3.1.1 Prerequisites

- Vault Radar binary installed at client machine.
- A valid Vault Radar license.
- A pre-commit configuration file.

For manually setting up the pre-commit step, you can either edit the pre-commit step file within your repository at `/.git/hooks/pre-commit` with the following command:

```
vault-radar scan git pre-commit "$@" || { exit "$?"; }
```

Or you can run the following command in the repository folder where you want to install the pre-commit hook:

```
vault-radar install git pre-commit-hook
```

This automatically creates the `/.git/hooks/pre-commit` file with the command shown in the first example. Using the installation command adds the scan command to the existing pre-commit script leaving any existing configuration untouched.

You also have to create a pre-commit configuration file. Creating a global configuration file at your `$HOME` directory can simplify this task significantly. For creating the configuration file, follow the steps in the Vault Radar git configuration documentation.

3.2 Pre-receive hooks (git server)

Use pre-receive hooks to centrally enforce scanning of commits.

3.2.1 Prerequisites

- A valid Vault Radar License (setup as an environment variable from a secret, or a file path).
- Installed Vault Radar binary by committing it to a repository or adding in a chroot environment.
- A pre-receive script.
- A Vault Radar configuration file.

By default Vault Radar looks for the files in a `/vault-radar` folder. If necessary, override this by setting the `VAULT_RADAR_CONFIG_PATH` environment variable to the path of the configuration file. The Vault Radar configuration file **must** contain a list of all risk types to scan at the `pre_receive_risk_allowlist` list. If this list is empty, the scan exits with a non-zero status.

```
{
   "pre_receive_skip_activeness": true,
   "pre_receive_fail_severity":"medium",
   "pre_receive_risk_allowlist": [
       "jwt_token",
       "GitHub personal access token"
   ]
}
```

Review the Configuration overview section in the Vault Radar documentation for more information.

The following is an example of how to set up a pre-receive hook using vault-radar for GitHub Enterprise Server (GHES).

```
#!/bin/bash

# get the full path to `git`
git=$(which git)
export GIT=$git

# Get the directory of the script
# this is needed to properly pass the location of the license file and config file to
    vault-radar
script_dir=$(dirname "$(realpath "$0")")

# Set the HOME environment variable to the githook user's home directory
# on GHES hook env HOME var is not set
# without it vault-radar will fail
export HOME=/home/githook
export VAULT_RADAR_LICENSE_PATH=$script_dir/vault-radar.hclic
export VAULT_RADAR_CONFIG_PATH=$script_dir/config.json

exec $script_dir/vault-radar scan git pre-receive
```

Once you create the repository with the files above, you will need to:

Go to your Enterprise settings > Hooks > Pre-receive hooks and click on *Add pre-receive hook*.

- Name the hook, e.g. *vault-radar*.
- Set the desired environment to execute when applicable.
- Choose the repository and the pre-receive script above.
- To enforce your script, select Use the **exit-status to accept or reject pushes**. Deselecting this option allows you to test the script while ignoring the exit-status value.
- To allow organization members with administrator or owner permissions to select whether they wish to enable or disable this pre-receive hook, select **Administrators can enable and disable this hook**.

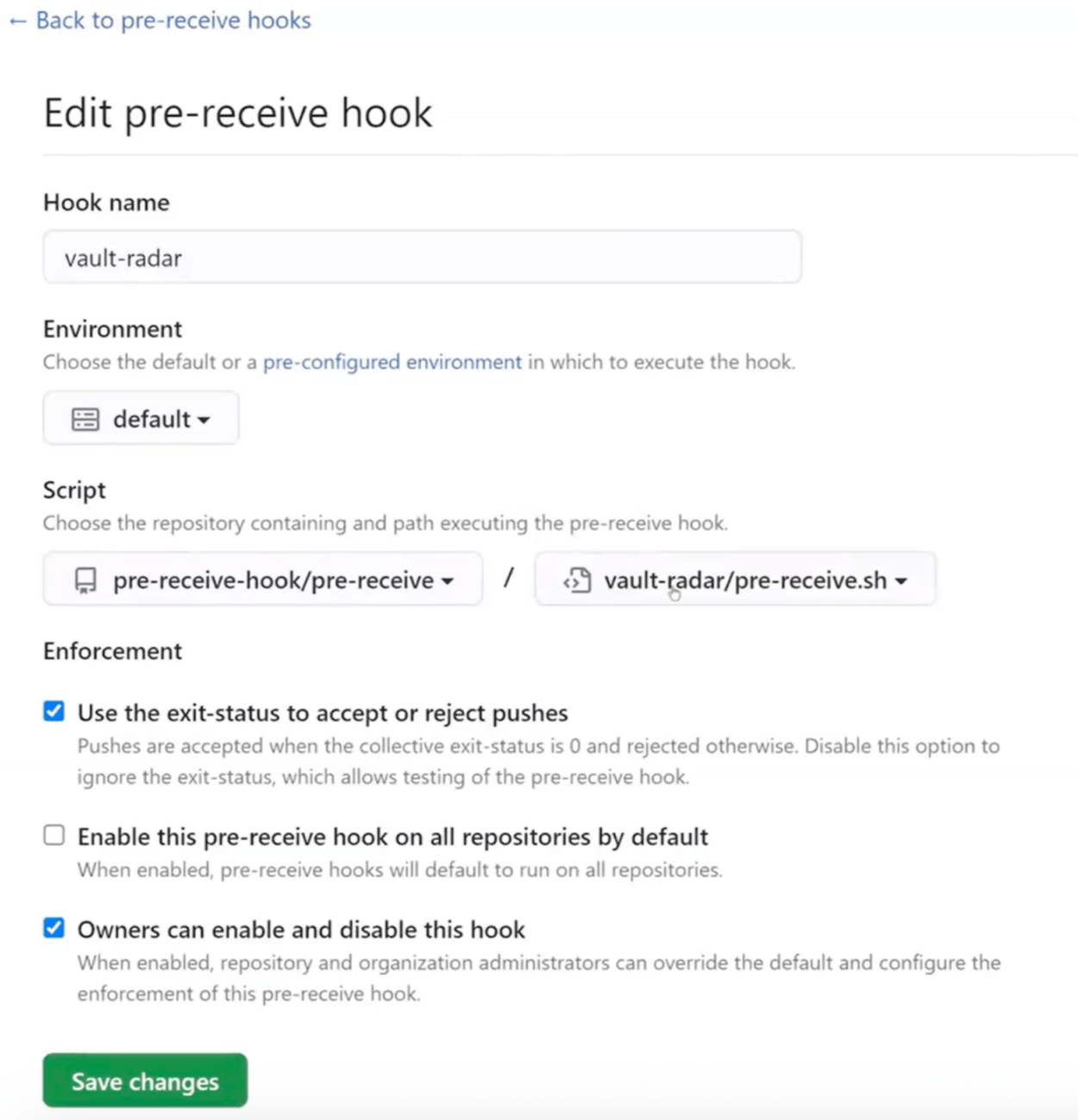


Figure 3: Pre-receive hook configuration interface

3.3 Pull request scanning

Follow the Pull request check policies tutorial in the Vault Radar docs.

3.4 CI/CD scanning

Use the `scan ci` command to scan content in a continuous integration workflow.

3.4.1 Prerequisites

- Vault Radar binary file.
- An HCP project level service principal with the Vault Radar Agent role assigned.

Use the `tip` scan to the latest commit on a branch, this is often used in pipeline steps following a commit. This approach only scans the contents of the latest commit and does not do a full rescan of the repository.

Use the `pr` scan when reviewing a change. Similar to the `tip` scan, it does not perform a full re-scan of the repository, but instead supports options to scan all the commits of the specified pull request.

> Tip
>
> PR here means pull request. This is a GitHub concept. The command works for any git based VCS. Anywhere we use the term PR, you can substitute Merge Request, Code Review, or any other terminology.

If the commits are very large (gigabytes of data) these commands could take additional time and require additional agent memory allocation in order to check out and scan the content. In the case of small to medium size commits, there should not be any need for improved agent hardware or additional computational power for Vault Radar scanning to be successful.

For a detailed example, please check the Example Vault Radar CI configurations in the docs.

3.5 Running Vault Radar CLI scan and meter commands

The scanning and metering commands are typically more comprehensive and scan all of the contents as well as the entire history of the resource. Similar to CI/CD scanning, the main bottleneck with the commands is the size of the scanned content.

Vault Radar does its best to efficiently scan contents but we are often at the mercy of the integration. Git resources, for example, are very efficient. We are able to compress the contents and scan only the incremental diffs making the memory burden generally low (for typical cases at least, if you know your commits again are unusually large this may not hold true). For resources like a Confluence Space, the integration does not provide a way to retrieve the contents efficiently. For every version of a page it must download and scan the whole page. Because of these differences it is important to have some understanding of the resources you intend to scan.

Scanning itself is not particularly CPU intensive. An increase of CPU resources could yield faster results, but we do not expect the improvements to be drastically significant. Additionally most of the commands are very network intensive with the exception of resources located on the host machine itself. Memory is generally the main constraint.

To help with scaling we provide this table describing considerations for the specific data source or resource.

Resource	Scanned Entity	Notes	Metering
AWS Parameter Store (Beta)	Parameter entry	Typically small	No
AWS S3 (Beta)	S3 Object	S3 Object can be very large	No
Git repository	Cloned repository	Typically small	Yes
Confluence	A version of a Confluence page	Confluence pages can be very large	Yes
Docker image (Beta)	Docker image	Typically small	No
File or folder (Beta)	File or folder	The contents are on the host already	No
Jira	Jira issue	Typically small	Yes

Resource	Scanned Entity	Notes	Metering
Slack (Beta)	Slack message	Typically small	No
HCP Terraform (Beta)	Terraform configuration	Typically small	No

3.6 CLI indexing commands

Vault Radar supports indexing AWS Parameter Store and Vault using the CLI. In both cases the CLI has to read the contents of each item in the data source.

Similar to scanning, the main consideration is the size of the largest entries in the data source. Vault entries by default can be up to 512 KiB and 4KB for AWS Parameter Store. Vault Radar creates the index as a file on the machine. The index is small, containing information about the location, a hash of the contents used as the index, and some metadata. The host machine must have enough memory to scan entries while constructing the index.

4 Remediation workflows

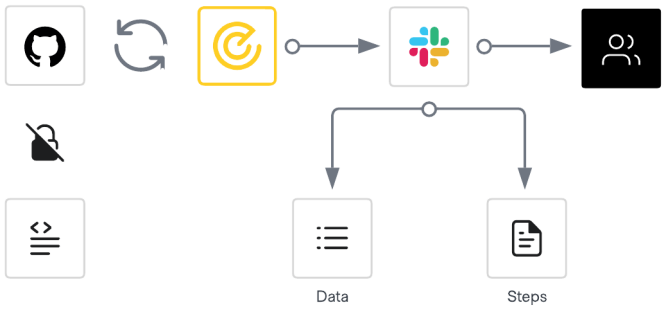


Figure 4: Vault Radar remediation workflow overview

Beyond secret detection, Vault Radar has a suite of features to help teams efficiently remediate issues found by Radar.

The features are:

- Notification integrations
- Resource based roles
- Remediation recommendations
- Remediation actions

Notification integrations and resource-based roles are configured by platform operators. For those, see the Administration Guide: Observability and alert management and Identity and access management. The remaining sections describe the remediation experience for developers and teams consuming Vault Radar.

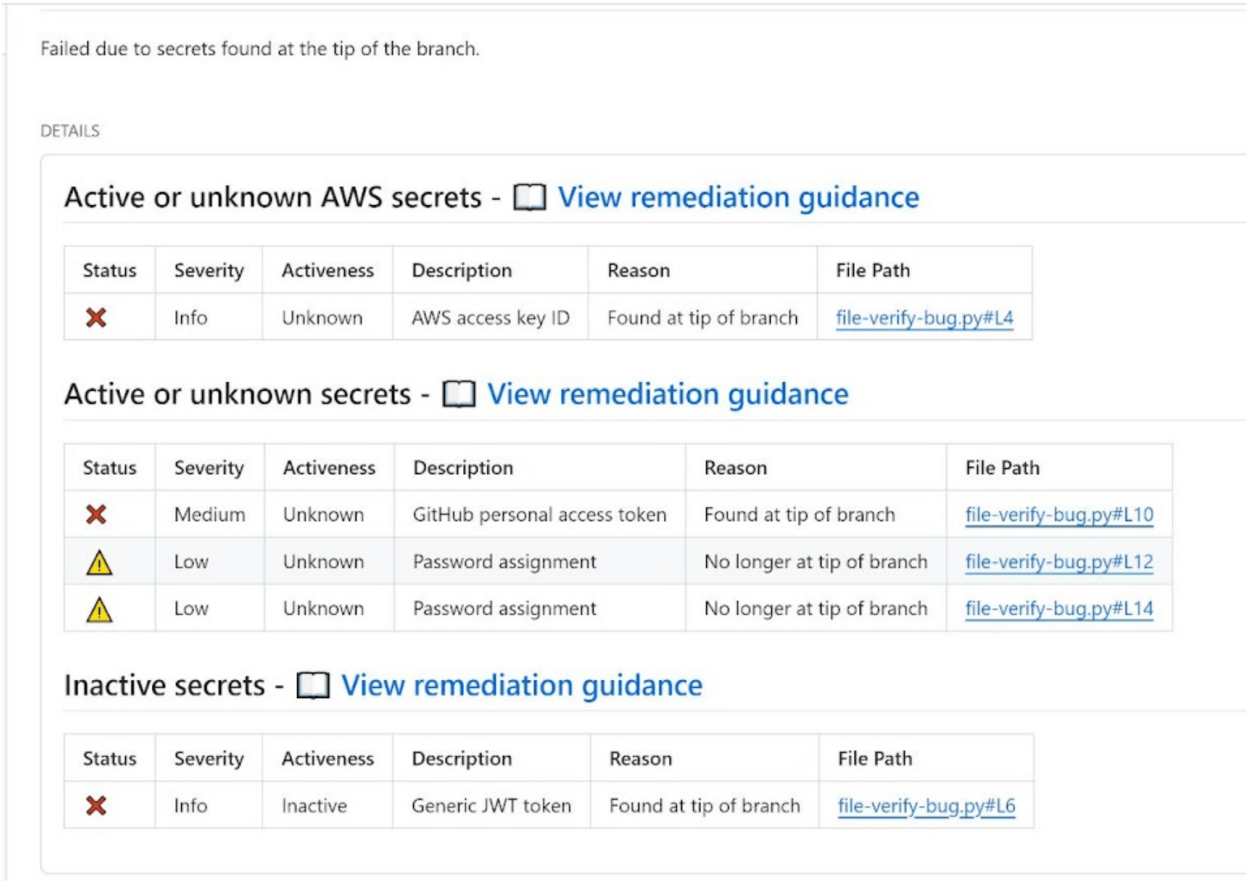
4.1 Remediation recommendations

Remediation documents provide recommendations to developers based on the type of secret found in code and the secret's activeness. There are currently four provided core remediation documents.

- Active or unknown AWS secrets
- Active or unknown GCP secrets
- Active or unknown secrets (general)
- Inactive secrets

Organizations can also create their own custom remediation documents and add the link to Vault Radar to make it available in-product.

Remediation recommendations are also accessible within GitHub pull requests when using the Vault Radar GitHub App.



Failed due to secrets found at the tip of the branch.

DETAILS

Active or unknown AWS secrets - View remediation guidance

Status	Severity	Activeness	Description	Reason	File Path
✖	Info	Unknown	AWS access key ID	Found at tip of branch	file-verify-bug.py#L4

Active or unknown secrets - View remediation guidance

Status	Severity	Activeness	Description	Reason	File Path
✖	Medium	Unknown	GitHub personal access token	Found at tip of branch	file-verify-bug.py#L10
⚠	Low	Unknown	Password assignment	No longer at tip of branch	file-verify-bug.py#L12
⚠	Low	Unknown	Password assignment	No longer at tip of branch	file-verify-bug.py#L14

Inactive secrets - View remediation guidance

Status	Severity	Activeness	Description	Reason	File Path
✖	Info	Inactive	Generic JWT token	Found at tip of branch	file-verify-bug.py#L6

Figure 5: GitHub pull request remediation recommendations

4.2 Remediation actions

Remediation actions allows Vault Radar administrators to setup a connection between Radar and Vault that allows development teams to easily store leaked secrets in the correct Vault locations so they can expedite the remediation steps.

For instructions on how to set up remediation actions with Vault, read the Copy leaked secrets into HashiCorp Vault documentation.

5 Changelog

5.1 2026-08

- Restructured legacy Vault Radar HVD content into the HVD 2.0 Vault Radar User Guide as part of the Installation / Administration / User guide migration.

6 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Andre Pimentel
- Cami Cano Gomez
- David Mills
- Mark Lewis